
II

STARTING WITH THE BRICKS: PROGRAMMING PARADIGMS

Software architecture begins with the code—and so we will begin our discussion of architecture by looking at what we’ve learned about code since code was first written.

In 1938, Alan Turing laid the foundations of what was to become computer programming. He was not the first to conceive of a programmable machine, but he was the first to understand that programs were simply data. By 1945, Turing was writing real programs on real computers in code that we would recognize (if we squinted enough). Those programs used loops, branches, assignment, subroutines, stacks, and other familiar structures. Turing’s language was binary.

Since those days, a number of revolutions in programming have occurred. One revolution with which we are all very familiar is the revolution of languages. First, in the late 1940s, came assemblers. These “languages” relieved the programmers of the drudgery of translating their programs into binary. In 1951, Grace Hopper invented A0, the first compiler. In fact, she coined the term *compiler*. Fortran was invented in 1953 (the year after I was born). What followed was an unceasing flood of new programming languages—COBOL, PL/1, SNOBOL, C, Pascal, C++, Java, ad infinitum.

Another, probably more significant, revolution was in programming *paradigms*. Paradigms are ways of programming, relatively unrelated to languages. A paradigm tells you which programming structures to use, and when to use them. To date, there have been three such paradigms. For reasons we shall discuss later, there are unlikely

to be any others.